

Role 3 – Data Format Analysis dan Python Parsing

1. Pendahuluan

Pada bagian ini, dilakukan analisis terhadap data hasil response dari API yang digunakan dalam praktikum Communication Protocol. Fokus utama adalah memahami struktur data (JSON/XML/Protobuf), mengidentifikasi field penting, serta melakukan proses parsing menggunakan Python agar data dapat diolah menjadi bentuk yang lebih terstruktur seperti tabel atau CSV.

2. Analisis Format Data

Data yang digunakan pada praktikum ini berupa format JSON (JavaScript Object Notation) yang umum digunakan dalam komunikasi API berbasis REST.

Contoh data response:

```
{  
  "id": 1,  
  "name": "Data",  
  "value": 123  
}
```

Dari struktur tersebut dapat dianalisis sebagai berikut:

- Id → bertipe integer, berfungsi sebagai identitas unik data
- Name → bertipe string, merepresentasikan nama atau label data
- Value → bertipe integer, berisi nilai numerik dari data

Jika data berbentuk lebih kompleks, JSON dapat memiliki:

- Nested object (objek di dalam objek)
- Array/List (kumpulan data)
- Key-value pair sebagai struktur utama

3. Proses Parsing dengan Python

Parsing dilakukan menggunakan Python dengan library json dan pandas untuk mengubah data mentah menjadi bentuk tabel.

Contoh script:

```
import json
```

```
import pandas as pd
```

```
# contoh data JSON
```

```
data_json = '''
```

```
[
```

```
  {"id": 1, "name": "Data", "value": 123},
```

```
  {"id": 2, "name": "Info", "value": 456}
```

```
]
```

```
'''
```

```
# parsing JSON
```

```
data = json.loads(data_json)
```

```
# ubah ke DataFrame
```

```
df = pd.DataFrame(data)
```

```
# tampilkan hasil
```

```
print(df)
```

```
# simpan ke CSV
```

```
df.to_csv("hasil_parsing.csv", index=False)
```

4. Hasil Output Parsing

Hasil parsing data menghasilkan tabel sebagai berikut:

```
[ ] import pandas as pd
    from datetime import datetime

    data = [
        {"ticker": "AAPL", "price": 175.5, "volume": 50000000, "timestamp": 1780747100},
        {"ticker": "MSFT", "price": 420.2, "volume": 23000000, "timestamp": 1780747120}
    ]

    df = pd.DataFrame(data)

    df["waktu_lokal"] = df["timestamp"].apply(
        lambda x: datetime.fromtimestamp(x).strftime("%Y-%m-%d %H:%M:%S")
    )

    df
```

	ticker	price	volume	timestamp	waktu_lokal
0	AAPL	175.5	50000000	1780747100	2026-06-06 11:58:20
1	MSFT	420.2	23000000	1780747120	2026-06-06 11:58:40

```
[ ] df.to_csv("hasil_parsing.csv", index=False)
```

Colab paid products - Cancel contracts here

Variables Terminal

Output ini juga disimpan dalam file:

Hasil_parsing.csv

Sehingga data dapat digunakan kembali untuk analisis lebih lanjut.

5. Penjelasan Proses Transformasi Data

Proses transformasi yang dilakukan adalah:

- 1) Mengambil data mentah (raw JSON response dari API)
- 2) Mengubah JSON string menjadi Python object menggunakan `json.loads()`
- 3) Mengonversi data menjadi DataFrame menggunakan `pandas`
- 4) Menyimpan hasil akhir ke format CSV

Alur ini menunjukkan bagaimana data dari API yang awalnya tidak terstruktur secara tabel dapat diubah menjadi format yang lebih mudah dianalisis.

6. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa:

- JSON merupakan format data yang fleksibel untuk komunikasi API
- Python memudahkan proses parsing dan transformasi data
- Data hasil API dapat diubah menjadi bentuk tabel untuk analisis lebih lanjut
- Proses parsing merupakan bagian penting dalam data engineering dan data science

Refleksi Pribadi:

Saya memegang Role 3 dalam kelompok, yaitu bertanggung jawab pada analisis data format dan parsing menggunakan Python. Tugas utama saya adalah menganalisis response API yang berbentuk JSON serta mengubahnya menjadi format tabel menggunakan `pandas`.

Dalam proses pengerjaan, saya membuat script Python untuk membaca data JSON, melakukan parsing, dan mengonversinya menjadi DataFrame. Hasil akhirnya kemudian saya simpan dalam bentuk file CSV sebagai bukti hasil pemrosesan data.

Kendala yang saya hadapi adalah saat menangani struktur JSON yang memiliki format list dan nested object, sehingga awalnya data tidak bisa langsung dikonversi menjadi tabel. Saya mengatasi masalah tersebut dengan mempelajari kembali struktur JSON dan menggunakan `json.loads()` sebelum melakukan konversi ke DataFrame.

Dari praktikum ini, saya memahami bahwa data dari API tidak selalu langsung dalam bentuk tabel, sehingga perlu proses parsing terlebih dahulu agar dapat dianalisis lebih lanjut. Saya juga memahami pentingnya `pandas` dalam pengolahan data karena mempermudah transformasi data ke bentuk yang lebih terstruktur.

Secara keseluruhan, saya merasa kontribusi saya dalam Role 3 sudah berjalan dengan baik karena berhasil menghasilkan script Python, output CSV, dan analisis struktur data sesuai kebutuhan tugas.